You prime the pump by loading the `old_head` value before you enter the loop ❶ and before you increase the external count on the loaded value ❷. If the `head` node is the same as the `tail` node, you can release the reference ❸ and return a null pointer because there's no data in the queue. If there is data, you want to try to claim it for yourself, and you do this with the call to `compare_exchange_strong()` ❹. As with the stack in listing 7.11, this compares the external count and pointer as a single entity; if either changes, you need to loop again, after releasing the reference ❻. If the exchange succeeded, you've claimed the data in the node as yours, so you can return that to the caller after you've released the external counter to the popped node ❺. Once both the external reference counts have been freed and the internal count has dropped to zero, the node itself can be deleted. The reference-counting functions that take care of all this are shown in listings 7.17, 7.18, and 7.19.

**Listing 7.17. Releasing a node reference in a lock-free queue**